

Day 03 Practice Assignments: Mutable Lists

Objective

Practice list declarations, indexing, slicing, modification methods, list operators, list comprehensions, and nested lists through interactive, scenario-based problems.

[!IMPORTANT] **Functions (def) have not been taught yet.** Write all assignments as procedural scripts. Prompt the user for input using `input()`, perform list processing, and print results using `print()`.

Part A: Easy Complexity (3 Exercises)

Exercise 1: The Wizard's Magic Bag

Scenario: A wizard has a magic bag containing a sequence of items: `["staff", "potion", "spellbook"]`. When the wizard steps through a magic portal, two things happen:

1. A new item enters the bag (prompts the user to input the item name to append to the end).
2. The oldest item in the bag (at index 0) is dissolved and ejected. Write a program to simulate this portal transition and print the final bag contents.

- **Sample Input:** (User inputs `"amulet"`)
- **Sample Output:**

```
Portal transition activated!
Ejected oldest item: staff
Current items in the magic bag: ['potion', 'spellbook', 'amulet']
```

Exercise 2: Movie Night Playlist

Scenario: You are organizing a movie marathon. You start with a playlist: `["Inception", "The Matrix", "Interstellar"]`. Prompt the user to enter the name of a movie they want to add.

- If the movie is already in the list, print `"Already added!"` and do not insert it.
- If it is not in the list, append it to the end of the list. Finally, sort the movie list alphabetically and print the updated playlist.
- **Sample Input:** `"Interstellar"`
- **Sample Output:**

```
Already added!
Alphabetical Playlist: ['Inception', 'Interstellar', 'The Matrix']
```

- **Sample Input:** `"Avatar"`

- **Sample Output:**

```
Added Avatar!  
Alphabetical Playlist: ['Avatar', 'Inception', 'Interstellar', 'The  
Matrix']
```

Exercise 3: The Cargo Train Scanner

Scenario: A train has wagons carrying different resources: ["coal", "iron", "gold", "coal", "timber", "coal"]. The train conductor wants to inspect the cargo. Write a program that prompts the user to enter a resource type (e.g., "coal" or "gold").

- Print the total number of wagons carrying that resource (using `.count()`).
- If the resource is on the train, print the index of the very first wagon carrying it (using `.index()`). If it is not found, print "Resource not found on train!".

- **Sample Input:** "coal"

- **Sample Output:**

```
Number of coal wagons: 3  
First coal wagon is at index: 0
```

- **Sample Input:** "oil"
- **Sample Output:** "Resource not found on train!"

Part B: Medium Complexity (5 Exercises)

Exercise 4: Nightclub VIP Queue

Scenario: A nightclub bouncer maintains a list of VIP guests who are allowed inside: ["Guido", "Esha", "Rajan", "Kishori"]. As guests arrive at the door, the bouncer prompts the user to enter their name.

- If the guest is on the VIP list, move them from their current position in the queue and insert them at the front of the queue (index 0).
- If the guest is not on the VIP list, print "Access denied. Not on the VIP list." and do not modify the list. Run this program in a loop. The loop should stop when the user types "exit". Print the updated queue state after each guest arrives.
- **Sample Walkthrough:**

```
Current VIP queue: ['Guido', 'Esha', 'Rajan', 'Kishori']  
Enter guest name: Rajan  
Rajan moved to the front!  
Current VIP queue: ['Rajan', 'Guido', 'Esha', 'Kishori']  
  
Enter guest name: Vinod
```

```
Access denied. Not on the VIP list.  
Current VIP queue: ['Rajan', 'Guido', 'Esha', 'Kishori']  
  
Enter guest name: exit
```

Exercise 5: The Spy's Word Reverser

Scenario: A secret agent wants to send an encrypted message. The encryption rule is simple: **reverse every word in the sentence, but keep the order of words unchanged**. Write a program that prompts the user for a sentence, splits it, uses a list comprehension to reverse the letters of each word, and joins them back together.

- **Sample Input:** "Meet me at midnight"
- **Sample Output:** "teeM em ta thgindim"

Exercise 6: Grading on a Curve

Scenario: A professor wants to adjust exam grades. Prompt the user to enter a list of space-separated test scores. Convert them to a list of integers. Using a **single list comprehension with conditionals**, apply the following curve rules:

- If a score is below 50, add 10 points.
- If a score is 50 or higher, add 5 points.
- The maximum possible score is capped at 100 (e.g., a score of 98 becomes 100, not 103). Print the original and the curved grades.
- **Sample Input:** "45 88 30 98 50"
- **Sample Output:**

```
Original: [45, 88, 30, 98, 50]  
Curved: [55, 93, 40, 100, 55]
```

Exercise 7: Treasure Map Coordinate Filter

Scenario: You have a list of coordinate pairs representing suspected treasure locations on a map: `coords = [[12, 5], [-3, 14], [8, -2], [15, 9], [-5, -6]]`. However, the treasure can only exist in the first quadrant of the map (where **both** the X coordinate and Y coordinate are strictly greater than zero (i.e., `x > 0` and `y > 0`)). Write a program that uses a list comprehension to filter the list and print only the valid coordinates.

- **Hardcoded Input:** `coords = [[12, 5], [-3, 14], [8, -2], [15, 9], [-5, -6]]`
- **Sample Output:** `[[12, 5], [15, 9]]`

Exercise 8: De-duplicating Shopping Cart

Scenario: An online shopping cart has duplicate items due to double-clicks: `["apple", "banana", "apple", "orange", "banana", "banana"]`. Write a program that processes the list and removes

all duplicate items, **but keeps the first occurrence of each item in its original order**. Print the cleaned cart.

- **Hardcoded Input:** `cart = ["apple", "banana", "apple", "orange", "banana", "banana"]`
 - **Sample Output:** `['apple', 'banana', 'orange']`
-

Part C: Difficult Complexity (2 Exercises)

Exercise 9: The Josephus Elimination Game

Scenario: A group of N soldiers (numbered 1 to N) stand in a circle. Starting from the first soldier, every K -th soldier is eliminated from the circle. The count continues with the next remaining soldier, moving clockwise. This process repeats until only one soldier remains. Write a program that prompts the user to enter N (number of soldiers) and K (elimination interval). Simulate the game using a list and print the order of eliminations and the final survivor.

- **Sample Input:** `N = 5, K = 2`
- **Sample Output:**

```
Soldier circle initialized: [1, 2, 3, 4, 5]
Eliminated soldier: 2 (Remaining: [1, 3, 4, 5])
Eliminated soldier: 4 (Remaining: [1, 3, 5])
Eliminated soldier: 1 (Remaining: [3, 5])
Eliminated soldier: 5 (Remaining: [3])
The sole survivor is: 3
```

Exercise 10: Snake Game Board Renderer

Scenario: Render a simple 2D text game board. Write a program that performs the following steps in sequence:

1. Creates a 5×5 grid filled with dots "." represented as a nested list.
2. Places a food item "F" at grid position `[2, 3]`.
3. Prompts the user to enter coordinate inputs: a `row` and a `col` (integers between 0 and 4) for the snake's head.
4. Places the snake's head "S" at the user-supplied coordinate `[row, col]`, overwriting the character at that position.
5. If the user-supplied coordinates are exactly `[2, 3]`, print the message "Yum! The snake ate the food!" (the snake "S" will occupy index `[2, 3]` on the printed board, overwriting the "F").
6. Prints the grid neatly line-by-line (each row's elements separated by spaces).

- **Sample Input:** (User inputs Row 0 and Column 3)
- **Sample Output:**

```
. . . S .
. . . . .
```

```
. . . F .  
. . . . .  
. . . . .
```

- **Sample Input:** (User inputs Row 2 and Column 3)
- **Sample Output:**

```
. . . . .  
. . . . .  
. . . S .  
. . . . .  
. . . . .  
Yum! The snake ate the food!
```